

Manual: Filtro por Persona Asignada en la Vista de Tareas

1. Objetivo del manual

Este manual explica como implementar un filtro por persona asignada a la tarea en una vista de React.

Incluye:

- Explicacion no tecnica (entender la idea).
 - Explicacion tecnica (como codificarlo).
 - Paso a paso detallado.
 - Ejemplos practicos.
 - Errores comunes y como solucionarlos.
 - Checklist final de verificacion.
-

2. Explicacion no tecnica

Piensa en una lista de tareas de hotel o mantenimiento:

- Una tarea la tiene Ana.
- Otra tarea la tiene Carlos.
- Otra puede estar sin asignar.

Cuando hay muchas tareas, a veces quieres ver solo las de una persona.

El filtro por persona asignada permite:

- Ver todas las tareas.
- Ver solo las de Ana.
- Ver solo las de Carlos.
- Ver solo las tareas sin asignar (si decides incluir esa opcion).

En palabras simples:

- Es como elegir un nombre en un menu y ver solo lo que corresponde a ese nombre.
-

3. Explicacion tecnica

En React, este filtro se compone de 4 partes:

1. Estado del filtro seleccionado.
 - Ejemplo: `selectedAssignee`.
2. Opciones del select.
 - Se generan desde `users` (lista de usuarios) o desde `tasks` (usuarios realmente presentes).

3. Condicion de filtrado en filteredTasks.

- Se compara assignedUserId de cada tarea con selectedAssignee.

4. Select en la UI.

- Un select controlado que cambia selectedAssignee.

4. Paso a paso completo

Paso 1: crear el estado del filtro

Agrega un estado nuevo:

```
const [selectedAssignee, setSelectedAssignee] = useState('Todos')
```

Significa:

- selectedAssignee: valor elegido actualmente.
- setSelectedAssignee: funcion para actualizar el filtro.
- 'Todos': valor inicial.

Paso 2: definir como leer el usuario asignado en una tarea

Tu backend puede enviar diferentes nombres de campo. Usa una funcion compatible:

```
const getTaskAssignedUserId = (task = {}) => task.assignedUserId ??  
task.usuarioAsignadoId ?? null
```

Con esto evitas errores si cambia el nombre del campo.

Paso 3: preparar opciones del filtro de personas

Opcion recomendada (desde users)

Si ya tienes `users` cargados en estado:

```
const assigneeOptions = useMemo(() => {  
  const sorted = [...users]  
    .filter((user) => user?.id !== null)  
    .sort((a, b) => {  
      const aName = `${a.firstName || a.nombre || ''} ${a.lastName ||  
a.apellidos || ''}`.trim() || a.email || ''  
      const bName = `${b.firstName || b.nombre || ''} ${b.lastName ||  
b.apellidos || ''}`.trim() || b.email || ''  
    })  
  return sorted.map((user) => {  
    const name = user.firstName || user.nombre || user.apellidos || user.email || ''  
    return {label: name, value: user.id || user.usuarioAsignadoId || ''}  
  })  
})
```

```

        return aName.localeCompare(bName, 'es')
    })

    const formatted = sorted.map((user) => ({
      id: String(user.id),
      nombre:
        `${user.firstName} || user.nombre || ''} ${user.lastName} ||
        user.apellidos || ''}`.trim() ||
        user.email ||
        `Usuario ${user.id}`
    }))

    return [{ id: 'Todos', nombre: 'Todos' }, ...formatted]
  }, [users])

```

Opcion alternativa (incluir Sin asignar)

Si quieres poder filtrar tareas sin persona:

```

return [
  { id: 'Todos', nombre: 'Todos' },
  { id: 'SIN_ASIGNAR', nombre: 'Sin asignar' },
  ...formatted
]

```

Paso 4: agregar el select en la interfaz

Dentro del bloque de filtros:

```

<div>
  <label htmlFor='task-assignee-filter' className='form-label mb-1'>Asignado</label>
  <select
    id='task-assignee-filter'
    className='form-select'
    style={{ maxWidth: '250px' }}
    value={selectedAssignee}
    onChange={(e) => setSelectedAssignee(e.target.value)}
    aria-label='Filtrar por persona asignada'
  >
    {assigneeOptions.map((user) => (
      <option key={user.id} value={user.id}>{user.nombre}</option>
    ))}
  </select>
</div>

```

Buenas practicas:

- id unico (no repetir ids de otros filtros).
- Select controlado con value y onChange.

Paso 5: aplicar la condicion en filteredTasks

Agrega la condicion del nuevo filtro:

```
const filteredTasks = useMemo(() => {
  const term = normalizeSearchText(searchTerm)

  return tasks.filter((task) => {
    const apartmentId = getTaskApartmentId(task)
    const statusId = getTaskStatusId(task)
    const assignedUserId = getTaskAssignedUserId(task)

    const matchesSearch = !term || getTaskSearchIndex(task).includes(term)
    const matchesApartment = selectedApartment === 'Todos' ||
String(apartmentId ?? '') === selectedApartment
    const matchesStatus = selectedStatus === 'Todos' || String(statusId ??
'') === selectedStatus

    const matchesAssignee =
      selectedAssignee === 'Todos' ||
String(assignedUserId ?? '') === selectedAssignee

    return matchesSearch && matchesApartment && matchesStatus &&
matchesAssignee
  })
}, [
  tasks,
  searchTerm,
  selectedApartment,
  selectedStatus,
  selectedAssignee,
  apartmentNameById,
  userNameById,
  statusNameById
])
```

Si agregaste opcion SIN_ASIGNAR:

```
const matchesAssignee =
  selectedAssignee === 'Todos' ||
(selectedAssignee === 'SIN_ASIGNAR' && assignedUserId == null) ||
String(assignedUserId ?? '') === selectedAssignee
```

Paso 6: limpiar tambien este filtro

En tu funcion de limpiar filtros:

```
const clearFilters = () => {  
  setSelectedApartment('Todos')  
  setSelectedStatus('Todos')  
  setSelectedType('Todos')  
  setSelectedAssignee('Todos')  
}
```

Paso 7: verificar que users se este cargando

El filtro por persona depende de que **users** tenga datos.

En **refreshAll**, debe existir esta llamada:

```
getUsers()
```

Y tambien:

```
if (usersResult.status === 'fulfilled') {  
  setUsers(Array.isArray(usersResult.value) ? usersResult.value : [])  
}
```

Si users esta vacio, el filtro no tendra opciones.

5. Ejemplo practico de funcionamiento

Escenario:

- Tarea 1: asignada a usuario id 10 (Ana).
- Tarea 2: asignada a usuario id 11 (Carlos).
- Tarea 3: sin asignar.

Comportamiento esperado:

1. Seleccionas Ana en el filtro.
2. La tabla muestra solo tareas con assignedUserId = 10.
3. Seleccionas Todos.
4. Vuelven a mostrarse todas.

6. Errores comunes y soluciones

1. Error: el select aparece vacio.

- Causa: users no se cargo o fallo getUsers.
 - Solucion: revisar refreshAll y fallback de users.
2. Error: el filtro no cambia resultados.
- Causa: no agregaste matchesAssignee en el return final.
 - Solucion: incluirlo en la condicion final.
3. Error: comparacion falla por tipos de dato (string vs number).
- Causa: selectedAssignee viene string del select y assignedUserId puede ser number.
 - Solucion: comparar ambos como string.
4. Error: filtrar por Sin asignar no funciona.
- Causa: no existe condicion especial para null.
 - Solucion: agregar caso SIN_ASIGNAR.
5. Error: boton Limpiar no resetea persona.
- Causa: falta setSelectedAssignee('Todos').
 - Solucion: agregarlo en clearFilters.
-

7. Checklist rapido de validacion

Antes de terminar, confirma:

- Existe selectedAssignee en estado.
- Existe select de persona asignada en la UI.
- assigneeOptions se construye con users.
- filteredTasks tiene matchesAssignee.
- clearFilters reinicia selectedAssignee.
- getUsers y setUsers funcionan correctamente.

Si todo esto esta correcto, el filtro por persona asignada deberia funcionar bien.

8. Recomendacion de arquitectura

Para mantener consistencia entre filtros:

- Usa siempre valores string en los selects.
- Convierte ids a string solo al comparar.
- Centraliza helpers de lectura (getTaskAssignedUserId) para evitar codigo duplicado.
- Mantiene un patron similar para filtros de Apartamento, Estado, Tipo y Asignado.

Asi tu vista crece sin volverse dificil de mantener.